

Bayesian Classification (scored task: 4 points)

Tasks (Labs 1-2)

1. Implementation (from scratch) of LDA, QDA and NB (Naive Bayes) methods for binary classification (classes 0 and 1):
 - Create three classes in Python: LDA, QDA, NB
 - Each class should have three methods:
 - (a) Method `fit(X,y)`, where `X` is a design matrix, containing in each row the values of features for a single observation and `y` is a vector containing values of binary target variable for observations. This method fits the model using training data. In the case of LDA, QDA and NB training the model includes estimation of the parameters (vectors of means, covariance matrices, prior probabilities).
 - (b) Method `predict_proba(Xtest)` which computes predicted posterior probabilities for class 1 for observations whose feature values are in rows of the matrix `Xtest`.
 - (c) Method `predict(Xtest)` which assigns the predicted class (0 or 1) for observations whose feature values are in rows of the matrix `Xtest`.
 - (d) Method `get_params` that returns a list containing the estimated parameters.
 - Consider the numerical stability. Is it better to compute the likelihood? Or the log-likelihood?
2. Comparison of LDA, QDA and NB methods on simulated data.
 - There are two parameters which will vary in experiments: a and ρ . Generate training and testing data in the following way.
 - **Scheme 1:** Each dataset contain $n = 1000$ observations, $p = 2$ features and a binary variable that is generated from the Bernoulli distribution with probability of success 0.5. Features of the observations from the class 0 are generated independently from a normal standard distribution (mean 0, variance 1). Features of the observations from the class 1 are generated independently from a normal distribution (mean a , variance 1). **LDA assumptions are satisfied.**
 - **Scheme 2:** Each dataset contain $n = 1000$ observations, $p = 2$ features and a binary variable that is generated from the Bernoulli distribution with probability of success 0.5. Features of the observations from the class 0 are generated from a two-dimensional normal distribution (mean 0, variance 1, correlation ρ). Features of the observations from the class 1 are generated from a two-dimensional normal distribution (mean a , variance 1, correlation $-\rho$). **LDA assumptions are not satisfied.**
 - Compare LDA, QDA, and NB for both schemes (compute accuracy on the testing set) for fixed value $\rho = 0.5$ and different values of $a = 0.1, 0.5, 1, 2, 3, 5$. Use at least 30 random splits with fixed train/test proportion (e.g., 70/30) and generate boxplots showing the values of accuracy for each method and each value of the parameter a . Save the results in the file `BayesianSimulatedData1.pdf`

- Compare LDA, QDA, and NB for both schemes (compute accuracy on the testing set) for fixed value $a = 2$ and different values of $\rho = 0, 0.1, 0.3, 0.5, 0.7, 0.9$. Repeat the experiment for different train/test splits and generate boxplots showing the values of accuracy for each method and each value of the parameter ρ . Save the results in the file `BayesianSimulatedData2.pdf`
- For one chosen setting of parameters (e.g. $a = 2, \rho = 0.5$) generate a scatter plot showing observations from training set. Mark observations belonging to different classes using two different colors and two different symbols. Draw curves that separate classes for LDA and QDA. Save the results in the file `BayesianSimulatedData3.pdf`

3. Theoretical Questions.

- Why might NB sometimes perform surprisingly well even when features are correlated?
- Accuracy may not be the best scoring rule for this model. If you had to evaluate probabilities, which score would you use and why?
- What happens to the covariance matrix estimator and the resulting decision boundary in QDA when the number of observations in some class is close to (or smaller than) p ?
- How would look the model that combines the LDA and NB assumptions?

4. Comparison of LDA, QDA and NB methods on real data.

- Choose 2 datasets which are available on e.g., UCI repository <https://archive.ics.uci.edu/ml/index.php> or OpenML repository <https://www.openml.org/> related to binary classification problem. Please only focus on datasets with numerical features.
- Compare LDA, QDA, and NB. Split data into training set and test set. Train the model on the train set and compute accuracy on the test set. Repeat the experiment for different train/test splits and generate boxplots showing the values of accuracy for each method. Save the results for two datasets in the file `BayesianReal.pdf`

Density estimation

Tasks (Lab 3):

1.
 - Generate a sample of size 200 from the mixture of two Gaussian distributions, i.e. from a density:

$$f(x) = 0.9 \cdot f_{N(5,1)}(x) + 0.1 \cdot f_{N(10,1)}(x).$$

- Use **kernel density estimator** to approximate the true, theoretical density f corresponding to the above distribution. Draw the density function corresponding to the theoretical distribution and the estimated function $\hat{f}_n$.
- Compute mean squared error (MSE)

$$\frac{1}{K} \sum_{i=1}^K [f(x_i) - \hat{f}_n(x_i)]^2,$$

where $x_1, \dots, x_K$ are iid. from $U([2, 12])$.

- Analyse how the error depends on sample size n . Generate a plot showing how the mean squared error changes with n .
 - Analyse the influence of different kernel functions as well as smoothing parameters. Generate curves for different kernels and different values of smoothing parameters. Try at least 3 kernels and 3 values of smoothing parameters. In addition, use one of the smoothing parameter estimation methods.
 - What is the effect of too small / too large bandwidth?
2. Generate sample $X_1, \dots, X_n$ of size $n = 200$ from Gaussian mixture model described above. Compare two methods (compute MSE for both of them):
 - (a) **Method 1:** Kernel density estimator using sample $X_1, \dots, X_n$
 - (b) **Method 2:** Using sample $X_1, \dots, X_n$, generate artificial sample $X'_1, \dots, X'_k$ (you can choose k much larger than n) corresponding to the kernel density as follows:
 - Generate i from uniform distribution on $\{1, \dots, n\}$
 - Generate ϵ from $N(0, 1)$
 - Set $X'_j = X_i + \epsilon \cdot h$

Compute kernel density estimator using modified sample $X'_1, \dots, X'_k$.

Interpret why the two methods should be close. Does increasing k change anything essential?

3. Select any dataset corresponding to binary classification problem with quantitative variables. Compare the accuracy of:

- Naive Bayes method (with kernel density estimator)
- Naive Bayes method (with Gaussian approximation of the density)
- Naive Bayes method (with discretization of quantitative features)
- LDA

Logistic Regression (scored tasks: 4 points)

Tasks (Lab 4):

1. Problem of linearly separable classes.

Dataset *earthquake.txt* corresponds to the problem of predicting seismic shocks (volcanic eruptions and nuclear explosions) (variable **popn**) based on two variables: **body** (deep wave magnitude) and **surface** (surface wave magnitude).

- Make a scatterplot for variables **body** and **surface**. Mark classes corresponding to observations.
- Fit logistic model without regularization, print estimated coefficients, estimated probabilities and compute log-likelihood function.
- Fit logistic model with ℓ_2 regularization, print estimated coefficients, estimated probabilities and compute log-likelihood function.

Compare the two fits and explain what happens when the classes are linearly separable. Why can the coefficients in the unregularized model become very large?

To the scatterplot, add the decision boundary for both models.

2. Simulation example.

- Generate data from logistic model:

$$y_i \sim \text{Bern}(p_i),$$

where

$$p_i = \frac{1}{1 + \exp[-(\beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \dots + \beta_5 x_{i,5})]},$$

for $i = 1, \dots, n$, $x_{i,1}, \dots, x_{i,5} \sim N(0, 1)$, $n = 50$. Parameters: $\beta_0 = 0.5$, $\beta_1 = \dots = \beta_5 = 1$. Fit a logistic regression model (with intercept) and calculate the estimators of the coefficients $\hat{\beta} = (\hat{\beta}_1, \dots, \hat{\beta}_5)$. Repeat the experiment $L = 100$ times and estimate the MSE (mean squared error):

$$MSE = E(\|\hat{\beta} - \beta\|^2),$$

$$\widehat{MSE} = \frac{1}{L} \sum_{i=1}^L \|\hat{\beta}^{(i)} - \beta\|^2,$$

where $\|\cdot\|$ is Euclidean norm, $\beta = (\beta_1, \dots, \beta_5)$ is vector of true parameters, $\hat{\beta}^{(i)}$ is a vector of estimated parameters in i -th experiment.

- Repeat the experiment for $n = 50, 60, 70, 80, 90, 100, 200, 300, \dots, 1000$ and make a plot showing how $\widehat{MSE}$ depends on n .
- Using the same simulated datasets (the real model has $\beta_0, \beta_1, \dots, \beta_5$), train the model based only on 3 variables (and intercept): $x_{i,1}, x_{i,2}, x_{i,3}$ and draw an analogous curve showing how the $\widehat{MSE}$ for $\beta = (\beta_1, \beta_2, \beta_3)$ depends on n . Compare both curves and comment on the effect of fitting a misspecified model with omitted relevant variables.

Evaluation methods

Setup:

1. Prepare two datasets:
 - (a) Choose one real dataset corresponding to a binary classification problem.
 - (b) Create an artificial dataset in the following way:

$$y_i \sim \text{Bern}(p_i),$$

where

$$p_i = \frac{1}{1 + \exp[-(\alpha + \beta^T x_i)]},$$

for $i = 1, \dots, n$, $x_i \sim N(0, I)$ and $\beta = (b, b, b, b, b, 0, \dots, 0)$ (there are 5 relevant variables and k irrelevant variables).

The parameters n, b, k , and α control, respectively, the sample size, signal strength, number of irrelevant variables, and class imbalance. We will vary them in simulations.

2. How will increasing α influence the imbalance in the dataset?

For a given α , what is the expected class proportion $\mathbb{P}(Y = 1)$? How does it determine the class imbalance?

3. Consider two classification models: logistic regression and classification tree. Which model do you expect to be more stable under repeated sampling?

Tasks:

1. For one chosen setting of parameters (e.g. $n = 1000$, $b = 1$, $k = 20$, $\alpha = 1$), estimate the misclassification error (using threshold 0.5) of both models using the following evaluation schemes:
 - (a) Refitting, i.e. use the whole dataset for both training and testing.
 - (b) 10-fold cross-validation.
 - (c) Bootstrap method.
 - (d) Bootstrap 0.632.

Compare the obtained estimates. Which method gives the most optimistic estimate? Which one seems the most reliable? Explain the differences.

2. Split data into training (50%) and validation (50%) sets. Draw ROC curve and precision-recall curve using the validation data for one of the models. Try different values of parameters for artificial dataset $n \in \{100, 1000\}$, $b \in \{0.5, 1\}$, $k \in \{5, 50\}$ and class imbalance $\alpha \in \{0, 2\}$. For which α value is the precision-recall curve more informative than the ROC curve?

3. Split data into training (50%) and validation (50%) sets. Fit the logistic regression model using training and calculate accuracy and balanced accuracy on validation data using classification rule

$$\mathbb{P}(y_i = 1|x_i) > t,$$

where t is a threshold. Draw plots showing how accuracy and balanced accuracy depend on $t \in [0, 1]$. On the accuracy plot, draw a line at $t = 0.5$; while for the balanced accuracy, draw a line at $t = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} y_i$. Why the accuracy / balanced accuracy tends to have a maximum around those points?

How would the optimal threshold change if false positives and false negatives had different costs?

Classification Trees

Tasks (Lab 6):

1. Classification tree.

Consider the dataset BreastCancer (`sklearn.datasets.load_breast_cancer`).

- (a) Fit a classification tree. Analyze how different parameters (`criterion`, `splitter`, `max_depth`, `min_samples_split`) affect the structure, depth, and size of the tree.
- (b) Draw the fitted tree and briefly interpret the first splits.
- (c) Prune the default tree using the cost-complexity criterion. Compare the pruned and unpruned trees in terms of size and predictive performance.

2. Bagging.

Implement your own version of the bagging algorithm. You may use available implementations of decision trees as base learners. Compare bagging with a single tree and comment on why bagging may improve stability and predictive performance.

3. Compare the following methods on 3 selected datasets:

- Bagging,
- Single tree,
- Random Forest.

Use default parameters. Compare the methods using the same train/test splits and report the results in a clear table. Briefly comment on the differences between the methods.

Logistic Regression: regularization (scored task: 4 points)

Tasks (Lab 7):

Recommended functions and packages: `glmnet` (library `glmnet` in R and Python), `LogisticRegression` (library: `scikit-learn` in Python).

1. Logistic regression with different regularizations: lasso, ridge, elastic net.

Consider high-dimensional dataset **prostate** corresponding to predicting the occurrence of prostate cancer using gene expression data. There are 102 observations (patients) and 6033 variables (genes). The response variable indicates whether a patient suffers from disease (prostate cancer, value: 1) or not (value: 0).

- Fit logistic regression models with different regularizations (lasso, ridge, elastic net). Print the values of parameters for different values of parameter λ .
- Make profile plots showing how the values of parameters depend on λ for different variables.
- Select the optimal value of λ using cross-validation.
- Compare the three regularization methods in terms of sparsity, shrinkage, and interpretability of the resulting model.
- Which regularization method sets coefficients exactly to zero? Which one tends to keep correlated variables together? Comment on the observed coefficient paths.
- Does the value of λ selected by cross-validation necessarily correspond to the best model for variable selection? Explain.

2. Performance of lasso for simulated data.

- Generate data from logistic model:

$$y_i \sim \text{Bern}(p_i),$$

where

$$p_i = \frac{1}{1 + \exp[-(\beta^T x_i)]},$$

for $i = 1, \dots, n$, $x_i \sim N(0, I)$ and

$$\beta = (1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0),$$

so there are 10 relevant variables and 10 irrelevant variables.

- Fit logistic model using lasso regularization (use `glmnet` package in R or Python). Use cross-validation to select the optimal value of parameter λ . Report which variables are selected as relevant by the method.

- Compute PSR (Positive Selection Rate)

$$PSR := \frac{|t \cap \hat{t}|}{|\hat{t}|}$$

and FDR (False Discovery Rate)

$$FDR := \frac{|\hat{t} \setminus t|}{|\hat{t}|},$$

where t is a set of true relevant variables and $\hat{t}$ is a set of variables selected by the model as relevant.

- Repeat the above steps $L = 100$ times and compute the averaged values of PSR and FDR.
- Investigate how the averaged values of PSR and FDR depend on the sample size (consider $n = 50, 100, 300, 500, 1000, 2000$); make a plot showing the dependence of PSR and FDR on n .
- Investigate how the averaged values of PSR and FDR depend on the number of irrelevant variables (consider values: 10, 50, 100, 200, 500) for fixed sample size $n = 300$.
- Can a model have good predictive performance and still poor variable selection quality? Compare the roles of PSR and FDR and explain what aspect of performance each of them measures.
- Analyze the robustness of the logistic regression model: generate y_i using

$$p_i = \Phi(\beta^T x_i),$$

where Φ is the distribution function of the standard Gaussian distribution. Repeat the remaining steps.

- Compare the results obtained for logistic and probit-type data generation. Does the lasso procedure still identify the important variables well? Which aspect seems more robust: prediction quality, PSR, or FDR?

Ensemble methods

Tasks (Labs 8-9):

1. Implementation of AdaBoost

- (a) Implement your own version of AdaBoost algorithm using trees as a base classifier – use available implementations of decision trees. Below is a description of AdaBoost algorithm:

Training:

- i. Define initial weights: $w_i = 1/n$, $i = 1, \dots, n$.
- ii. For $k = 1, \dots, B$:
 - A. Build classifier f_k with weights w_i .
 - B. Compute weighted classification error for the k -th classifier:

$$\varepsilon_k = \sum_{i=1}^n I[f_k(x_i) \neq y_i] w_i.$$

- C. If $\varepsilon_k \notin (0, 0.5)$ then handle this case appropriately and state clearly what your implementation should do.
- D. Compute scaling factor:

$$\beta_k = \frac{\varepsilon_k}{1 - \varepsilon_k}.$$

- E. Add a pair (f_k, β_k) to the ensemble.
- F. For $i = 1, \dots, n$:
If $f_k(x_i) = y_i$ (the example is correctly classified), then set $w_i = w_i \beta_k$.
- G. Normalize weights:

$$w_i = \frac{w_i}{\sum_{j=1}^n w_j}.$$

Prediction:

- Prediction for a new observation x :

$$\hat{y}(x) = \arg \max_y \sum_{k=1}^B I[f_k(x) = y] \log \left(\frac{1}{\beta_k} \right).$$

- Weighted voting.
 - Classifiers with smaller errors receive larger weights in the final vote.
- iii. In your implementation in point C of training, define clearly what should happen when $\varepsilon_k = 0$ or when $\varepsilon_k \geq 0.5$. Explain why these cases are problematic for AdaBoost.
 - iv. Compare AdaBoost with base trees of depth 1 and with deeper trees. Comment on how the depth of the base learner influences the results and time of training.
 - v. Theoretical questions:
 - A. Explain why we want a classifier with a smaller ε_k to receive a larger weight in the final prediction.
 - B. Assume that 999 classifiers have error $\varepsilon_k = 0.4$ and one classifier has error $\varepsilon_{1000} = 10^{-8}$. Compute the corresponding voting weights in AdaBoost and comment on whether one almost perfect classifier dominates the final decision.

C. Why are very simple base classifiers, such as decision stumps, often sufficient in AdaBoost?

2. (a) Compare the following ensemble methods:

- Single tree
- Bagging
- Boosting (AdaBoost), your implementation
- Gradient Boosting
- XGBoost
- Random Forest

(b) Consider one real dataset and one artificial dataset. You can choose the real dataset from, e.g., the UCI repository. The real dataset should correspond to a classification problem and should be of moderate size. You may use one of the datasets from the previous labs.

For the artificial dataset:

- Generate $X_1, X_2, \dots, X_{10} \sim N(0, 1)$.
- Denote by $\chi_{10}^2(0.5)$ the median of the chi-squared distribution with 10 degrees of freedom.
- Set $Y = 1$ if

$$\sum_{j=1}^{10} X_j^2 > \chi_{10}^2(0.5)$$

and $Y = -1$ otherwise.

- Generate training data of size 2000 and testing data of size 10000.

- (c) Why do we expect Y to be balanced in this artificial dataset?
- (d) For the artificial dataset, describe the shape of the Bayes decision boundary. Which of the considered methods should be better suited to approximate such a boundary, and why?
- (e) Make plots showing how the **training** and **testing** error change with the number of iterations/trees for boosting, bagging, Random Forest, Gradient Boosting, and XGBoost.
- (f) Compare the methods using the same train/test split for all of them.
- (g) For each method, briefly comment whether the improvement over a single tree seems to come mainly from variance reduction, bias reduction, or both.
- (h) Explain why Random Forest may outperform Bagging, although both methods average many trees.

Feature Selection

Tasks (Lab 10):

1. Data generation. Generate datasets as follows. Consider 3 parameters: n (size of the training data), p (number of all features) and k (number of significant features).

(a) Dataset 1:

- Generate independent features $X_1, X_2, \dots, X_p \sim N(0, 1)$.
- Denote by $\chi_k^2(0.5)$ median of chi squared distribution with k degrees of freedom.
- Set $Y = 1$ if $\sum_{j=1}^k X_j^2 > \chi_k^2(0.5)$ and $Y = 0$ otherwise.
- Generate training data of size n .

(b) Dataset 2:

- Generate independent features $X_1, X_2, \dots, X_p \sim N(0, 1)$.
- Set $Y = 1$ if $\sum_{j=1}^k |X_j| > k$ and $Y = 0$ otherwise.
- Generate training data of size n .

Are the relevant variables, $X_1, X_2, \dots, X_k$, expected to be detectable by simple marginal correlation with Y ? Why or why not?

2. Comparison of feature selection/feature ranking methods. Consider the following methods:

- Variable importance measures based on Random Forest (mean decrease in impurity and permutation based measure).
- Boruta algorithm (you can use R package `Boruta` or Python package `boruta_py`).

Tasks:

- (a) Check if the considered algorithms assign the highest variable importance scores to the significant variables.
- (b) Try different values of n , p and k . First, you can use values: $n = 500$, $p = 50$ and $k = 10$.
- (c) Repeat data generation process $L = 50$ times and estimate the probability of successful feature recovery, i.e. compute the fraction of simulations in which the significant variables are selected as relevant or are placed among the top k variables in the ranking list. Take smaller L if simulations are too expensive computationally.
- (d) Fix $n = 200$, $p = 500$, $k = 20$. Let t denote the number of top-ranked features according to the considered feature importance measure. Generate an independent test set. Train the classifier (e.g. random forest) using t top-ranked features and analyze how the test classification accuracy depends on $t = 5, 10, 15, 20, 50, 100, 200, 300, 400, 500$. Generate a plot showing the dependence.
- (e) Explain the role of shadow variables in Boruta. Why is comparison with the best shadow variable more meaningful than simply checking whether an importance score is positive?

Support Vector Machines (scored task: 4 points)

Tasks (Lab 11):

1. Linear SVM and logistic regression.

Generate a two-dimensional linearly separable dataset using the following code. The dataset is constructed in such a way that each class contains a large Gaussian cluster and a small group of observations close to the separating boundary.

```
import numpy as np
import matplotlib.pyplot as plt

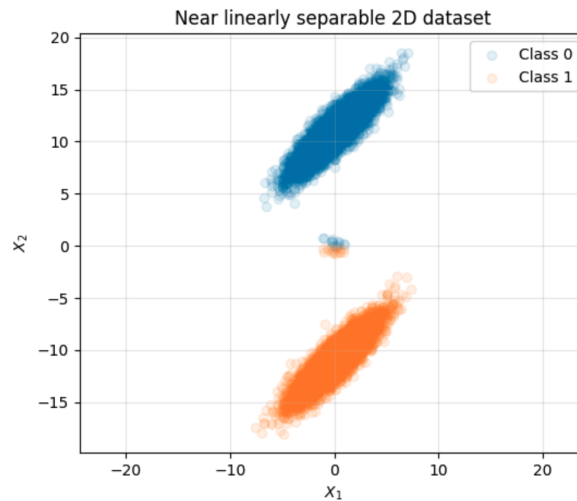
np.random.seed(42)

n_strip = 16
n_blob = 6000

# Class 0
x0_1 = np.hstack([
    np.random.normal(scale=0.6, size=(n_strip, 1)),
    np.random.uniform(0.3, 0.8, size=(n_strip, 1))
])
x0_2 = np.random.multivariate_normal(
    mean=[0, 10],
    cov=np.array([[4, 3.5], [3.5, 4]]),
    size=n_blob
)
X0 = np.vstack([x0_1, x0_2])
y0 = np.zeros(n_strip + n_blob)

# Class 1
x1_1 = np.hstack([
    np.random.normal(scale=0.6, size=(n_strip, 1)),
    np.random.uniform(-0.8, -0.3, size=(n_strip, 1))
])
x1_2 = np.random.multivariate_normal(
    mean=[0, -10],
    cov=np.array([[4, 3.5], [3.5, 4]]),
    size=n_blob
)
X1 = np.vstack([x1_1, x1_2])
y1 = np.ones(n_strip + n_blob)

X = np.vstack([X0, X1])
y = np.concatenate([y0, y1])
```



Before fitting the models, predict: will the SVM and logistic regression decision boundaries be similar or different? Which observations do you expect to influence each method most strongly?

Split the dataset into training and testing sets. Then fit two linear classifiers:

- logistic regression,
- linear SVM.

Before fitting the models, standardize the predictors.

For both methods:

- (a) compute training and testing accuracy,
- (b) draw the decision regions,
- (c) compare the obtained decision boundaries.

For the linear SVM additionally draw:

- the separating hyperplane,
- the margin lines,
- the support vectors.

Answer the following questions:

- (a) Both methods use linear decision boundaries. Why are the boundaries different?
- (b) Which observations become support vectors?
- (c) Are support vectors necessarily misclassified observations?
- (d) Does linear SVM estimate posterior probabilities $P(Y = 1 \mid X = x)$?
- (e) Why does the SVM boundary depend mainly on observations close to the separating boundary, while logistic regression is influenced by all observations?
- (f) How does the SVM boundary depend on the C parameter? Consider $C \in \{0.01, 0.1, 1, 10, 100\}$.
- (g) For each value of C , report the number of support vectors.

2. Kernel SVM.

Generate a two-dimensional nonlinear binary classification dataset using the following code.

```
from sklearn.datasets import make_moons

X, y = make_moons(
    n_samples=300,
    noise=0.25,
    random_state=123
)
```

Split the dataset into training and testing sets. Before fitting the models, standardize the predictors.

Fit the following classifiers:

- logistic regression,
- logistic regression with polynomial features of degree 2,
- logistic regression with polynomial features of degree 3,
- linear SVM,
- SVM with poly kernel (use $C = 10$, $coef0 = 10$ and $d \in \{2, 3, 4\}$),
- SVM with RBF kernel.

For each method:

- (a) compute training and testing accuracy,
- (b) draw the decision regions,
- (c) compare the shape of the decision boundary.

For the SVM with RBF kernel fix $C = 1$. Fit the model for several values of the parameter **gamma**:

$$\gamma \in \{0.01, 0.1, 1, 10, 100\}.$$

For each value of **gamma**:

- (a) compute training and testing accuracy,
- (b) draw the decision regions,
- (c) compare the complexity of the decision boundary.

Answer the following questions:

- (a) Why do ordinary logistic regression and linear SVM perform poorly on this dataset?
- (b) Does the logistic regression with polynomial features of degree 3 produce a proper decision boundary for this task?
- (c) What does the RBF kernel allow the SVM to do?
- (d) What happens when **gamma** is very small?
- (e) What happens when **gamma** is very large?
- (f) Compare logistic regression with polynomial features and SVM with RBF kernel. In what sense are these approaches similar, and in what sense are they different?

Regression I

Tasks (Lab 12):

1. Generate artificial data using the following model:

$$y = g(x) + \varepsilon,$$

where

$$g(x) = 4.26(e^{-x} - 4e^{-2x} + 3e^{-3x}),$$

and $\varepsilon \sim N(0, 0.1^2)$. Assume that x_i are generated independently from the uniform distribution on $[0, 4]$.

Generate both a training set and an independent test set.

2. Implement from scratch the Nadaraya-Watson kernel regression estimator:

$$\hat{g}_h(x) = \frac{\sum_{i=1}^n K\left(\frac{x-x_i}{h}\right) y_i}{\sum_{i=1}^n K\left(\frac{x-x_i}{h}\right)}.$$

Use at least the Gaussian kernel. You may also compare it with another kernel.

3. Compare the performance of two regression methods:
 - Nadaraya-Watson kernel regression,
 - smoothing splines regression.
4. For the Nadaraya-Watson estimator, compare several values of the bandwidth parameter h . For smoothing splines, compare several values of the smoothing parameter. Explain what happens when the fitted curve is too rough or too smooth. Relate your observations to the bias-variance tradeoff.
5. Visualize the results. Prepare a scatter plot of the training data and draw the curves corresponding to:
 - the true regression function $g(x)$,
 - the fitted Nadaraya-Watson estimators,
 - the fitted smoothing splines.

6. For every considered method and value of the smoothing parameter, compute two types of errors:

- function estimation error:

$$MSE_g = \frac{1}{n} \sum_{i=1}^n [g(x_i) - \hat{g}(x_i)]^2,$$

- prediction error:

$$MSE_y = \frac{1}{n} \sum_{i=1}^n [y_i - \hat{g}(x_i)]^2.$$

Compute both errors on the training set and on an independent test set.

Explain the difference between these two quantities. Which one is available only because this is a simulation experiment? Which one would be available for a real regression dataset?

7. Explore how the test error depends on the training sample size n . Consider several values of n , repeat the experiment several times for each value, and generate a plot showing the average test error as a function of n .

Does increasing n help equally for all smoothing parameter values? Explain why, for the Nadaraya-Watson estimator, the bandwidth h should usually decrease as n increases. What can go wrong if h is kept too large? What can go wrong if h decreases too quickly?

8. Which method is more local and which one is more global? How does a single new observation affect the fitted curve in Nadaraya-Watson regression and in smoothing spline regression?
9. Based on your experiments, explain why choosing the smoothing parameter is a model selection problem. Would you choose the parameter using training error, test error, or cross-validation? Justify your answer.

Semi-supervised learning

Tasks (Lab 13):

1. Consider any dataset corresponding to binary classification. For example, you can generate synthetic dataset using function `make_moons` from scikit-learn library.
2. Split data into training and testing subsets.
3. Choose randomly g observations in the training data as labeled. Remove the labels for the remaining observations in the training data.
4. Selected Semi supervised learning methods are implemented in scikit-learn library: https://scikit-learn.org/stable/modules/semi_supervised.html
5. Compare 4 methods:
 - (a) **Naive Method:** learn classifier using only g labelled examples in training data.
 - (b) **Self-training**
 - (c) **Label propagation**
 - (d) **Label spreading**

SSL methods (**Self-training**, **Label propagation** and **Label spreading**) use both labelled and unlabelled examples for training.
6. Use SVC (Support Vector Classifier) as a base classifier.
7. Compute accuracy, balanced accuracy, F measure and ROC AUC for the methods on testing data.
8. Analyse how the value of g affects the results.

Multi-label Classification (scored task: 4 points)

Tasks (Lab 14):

Introduction

Multi-label classification is a classification problem in which each observation may belong to several classes simultaneously. For an observation with feature vector x , the response is a binary vector

$$Y = (Y_1, \dots, Y_K) \in \{0, 1\}^K.$$

This differs from ordinary multi-class classification, where each observation belongs to exactly one class.

Example: Binary classification/Multi-class classification/Multi-label classification:



- **Binary classification:** Is there a mountain in the image? yes/no.
- **Multi-class classification:** Which single object is shown: mountain, desert, or car?
- **Multi-label classification:** Which labels are present: mountain, trees, forest, lake, sea, glacier? **Answer:** $Y = (1, 1, 1, 0, 0, 1)$.

Other examples include medical diagnosis, music tagging, document classification, and recommendation systems.

Data and starting code

```
import numpy as np

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.multioutput import ClassifierChain

from sklearn.metrics import accuracy_score
from sklearn.metrics import hamming_loss
from sklearn.metrics import jaccard_score

X, Y = fetch_openml("emotions", version=4, return_X_y=True)
Y = (Y == "TRUE").astype(int)

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

Technical note. Check your version of scikit-learn. Up to version 1.6.1, ClassifierChain
uses the argument base_estimator. In newer versions, this argument is replaced by estimator.

import sklearn
print(sklearn.__version__)
```

Task 1: Preliminary analysis of labels

Before fitting models, analyze the label matrix Y .

1. How many labels are there?
2. What is the empirical frequency of each label?
3. How many active labels does one observation have on average?
4. Which pairs of labels are most strongly correlated?
5. Are the labels approximately independent? Why does this matter for multi-label classification?

Task 2: Standard multi-label methods

Compare the following methods using logistic regression as the base classifier.

1. Binary Relevance (BR)

Binary Relevance fits one binary classifier per label:

$$Y_k \leftarrow X, \quad k = 1, \dots, K.$$

You may use

```
OneVsRestClassifier(LogisticRegression(solver="liblinear")).
```

2. Classifier Chains (CC)

Classifier Chains fit binary classifiers sequentially. For a chosen label order,

$$Y_1 \leftarrow X, \quad Y_2 \leftarrow X, Y_1, \quad \dots, \quad Y_K \leftarrow X, Y_1, \dots, Y_{K-1}.$$

At prediction time, previously predicted labels are used as additional input variables for later classifiers.

```
ClassifierChain(order="random", base_estimator=LogisticRegression(solver="liblinear")).
```

Test at least 10 different random label orders.

3. Ensemble of Classifier Chains (ECC)

An Ensemble of Classifier Chains fits several classifier chains with different random label orders and combines their predictions. Use at least $M = 20$ chains and combine predictions by majority voting:

$$\hat{Y}_{ik}^{ECC} = \mathbb{1} \left(\frac{1}{M} \sum_{m=1}^M \hat{Y}_{ik}^{(m)} \geq 0.5 \right).$$

Evaluation metrics

Let $Y^{(i)} = (Y_1^{(i)}, \dots, Y_K^{(i)})$ be the true label vector and let $\hat{Y}^{(i)} = (\hat{Y}_1^{(i)}, \dots, \hat{Y}_K^{(i)})$ be the predicted label vector.

Use the following metrics:

$$\text{SubsetAccuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left\{ Y^{(i)} = \hat{Y}^{(i)} \right\},$$

$$\text{HammingScore} = \frac{1}{NK} \sum_{i=1}^N \sum_{k=1}^K \mathbb{1} \left\{ Y_k^{(i)} = \hat{Y}_k^{(i)} \right\},$$

$$\text{JaccardScore} = \frac{1}{N} \sum_{i=1}^N \frac{\left| \{k : Y_k^{(i)} = 1\} \cap \{k : \hat{Y}_k^{(i)} = 1\} \right|}{\left| \{k : Y_k^{(i)} = 1\} \cup \{k : \hat{Y}_k^{(i)} = 1\} \right|}.$$

Subset accuracy is strict, Hamming score is label-wise, and Jaccard score focuses on active labels.

```
def evaluate(name, Y_true, Y_pred):
    subset_acc = accuracy_score(Y_true, Y_pred)
    hamming = 1 - hamming_loss(Y_true, Y_pred)
    jacc = jaccard_score(Y_true, Y_pred, average="samples")
    print(
        f"{name:5} {subset_acc:.3f} {hamming:.3f} {jacc:.3f}"
    )
```

Important. Jaccard score treats label 1 as the active label. If the meaning of 0 and 1 were swapped, the Jaccard score would change. In contrast, subset accuracy and Hamming score are symmetric with respect to swapping 0 and 1.

Task 3: Circular Chain Classifier

Implement your own method inspired by the **Circular Chain Classifier (CCC)**.

For each label Y_k , fit a binary classifier using both the feature vector X and all remaining labels Y_{-k} :

$$\hat{p}_k(x, y_{-k}) \approx P(Y_k = 1 \mid X = x, Y_{-k} = y_{-k}).$$

Equivalently, for every $k = 1, \dots, K$, fit

$$Y_k \leftarrow X, Y_1, \dots, Y_{k-1}, Y_{k+1}, \dots, Y_K.$$

During training, the true labels are known. During prediction, the true labels are not known and must be estimated iteratively.

Training

For each label k :

1. remove column Y_k from the label matrix;
2. append the remaining label columns Y_{-k} to the feature matrix X ;
3. fit a binary classifier predicting Y_k from (X, Y_{-k}) .

Use logistic regression as the base classifier.

Prediction

For a new observation, initialize a label vector

$$\hat{y}^{(0)} \in \{0, 1\}^K.$$

Try three initializations: random initialization, all zeros, and BR initialization.

Then choose a permutation of labels $\sigma = (\sigma(1), \dots, \sigma(K))$ and update labels sequentially:

$$\hat{y}_{\sigma(j)} \leftarrow \mathbb{1}(\hat{p}_{\sigma(j)}(x, \hat{y}_{-\sigma(j)}) \geq 0.5), \quad j = 1, \dots, K.$$

Repeat full passes through all labels until no label changes after a full pass. To avoid infinite loops, set for example

$$\text{max_iter} = 20.$$

The procedure is heuristic. It may depend on initialization and update order. If stabilization is not reached within `max_iter`, return the last iterate and report that convergence was not achieved.

Experiments

1. Implement functions `fit_ccc` and `predict_ccc`.
2. Compare CCC with random initialization, zero initialization, and BR initialization.
3. Check whether the result depends strongly on the random initialization.
4. Check whether the result depends strongly on the permutation σ .
5. Compare CCC with BR, CC, and ECC using the same evaluation metrics.
6. Report the average number of iterations needed until stabilization.

Questions

Answer the following questions briefly.

1. What label-dependence assumption is made by Binary Relevance?
2. Why can the order of labels influence Classifier Chains?
3. Why should ECC be more stable than a single Classifier Chain?
4. Why is subset accuracy usually much smaller than Hamming score?
5. Why is Jaccard score often more informative than Hamming score when active labels are rare?
6. Why can CCC be viewed as searching for a self-consistent label vector?
7. Does CCC define one coherent joint model

$$P(Y_1, \dots, Y_K \mid X = x),$$

or only a collection of conditional models? Why is this distinction important?